Java OOP Laboratory Exercise

Campus Café Ordering System

Topics: Conditional Statements, Classes and Objects, Constructors, Encapsulation, Polymorphism, and Abstraction

Name: Villanueva, Alexander S.
Baena, Charlie Angelo M.
Section: BSINT2-SEC-E
Date: November 10, 2026

1. Why This Project?

The exercise asks us to build a console ordering system for a university café. A café is a good fit for object-oriented programming because everything in it is already a “thing” with data and behavior: a customer who earns points, a menu item with a price, an order that ties them together. The menu also has three kinds of items (drinks, pastries, meals) that share common traits but are priced differently, which is exactly the situation inheritance, abstraction, and polymorphism are meant for. Rules such as the student discount, the three-item limit, and the voucher floor of 0 give us natural places to practice if and if-else statements. The no-collections rule forced us to think carefully about how objects are stored and how they interact.

2. How We Built It

Step 1 - Design first. We read the specification and listed the seven classes and how they relate: Beverage, Pastry, and Meal extend the abstract MenuItem; an Order holds one Customer and up to three MenuItem objects; CafeApp drives everything.

Step 2 - Parent class. We wrote MenuItem with its abstract computePrice() and a validated setBasePrice().

Step 3 - Subclasses. Each subclass overrides computePrice() and getDescription() with its own rule.

Step 4 - Customer and Order. We added overloaded constructors and addPoints(), then the order logic.

Step 5 - Test. In CafeApp we reproduced the sample scenario and checked every computed value against the expected output.

Step 6 - Document. We added Javadoc and inline comments, and compiled with javac.

3. Concept Explanation

Concept	Where and why we used it
if Statement	Pastry.computePrice() adds the warming fee only when warmed; Meal.computePrice() adds the upsize fee only when upsized.
if-else Statement	MenuItem.setBasePrice(), Beverage.computePrice(), Order.addItem(), and Order.computeTotal() each choose between outcomes, such as rejecting a negative price or filling the first empty slot.
Classes and Objects	Seven classes model the café; CafeApp creates objects from them (customers, items, orders).

Concept	Where and why we used it
Attributes and Behaviors	Every class has instance variables (e.g. name, size) and instance methods (e.g. computePrice(), addItem()).
Object Interaction	Order stores a Customer and MenuItem objects, calls their methods, and printReceipt() updates the customer's loyalty points.
Naming Conventions	PascalCase classes, camelCase members, and UPPER_SNAKE_CASE constants such as WARMING_FEE and UPSIZE_FEE.
Default Constructor	MenuItem(), Beverage(), Pastry(), Meal(), and Customer() give each object safe starting values.
Parameterized Constructor	All MenuItem subclasses, Customer, and Order accept values at creation so objects are ready to use.
Constructor Overloading	Customer has three constructors: (), (name), and (name, student), so callers supply only what they know.
public / private / protected	Fields are private or protected (name and basePrice are protected so subclasses can price themselves); behavior is public.
Getters, Setters, Data Hiding	No public fields. Setters validate input, e.g. negative prices, invalid sizes, and negative points are rejected.
Method Overloading	Customer.addPoints(int) vs addPoints(double), and Order.computeTotal() vs computeTotal(double) - same name, different parameters.
Method Overriding	Beverage, Pastry, and Meal override computePrice() and getDescription() with @Override.
Abstraction	MenuItem is abstract with an abstract computePrice(), so no generic item can be created and every subclass must define a price.
Polymorphism in action	Order.computeSubtotal() calls computePrice() on MenuItem references; Java runs the correct subclass version for each item.

4. Program Output

Output of java CafeApp. (Replace with your own screenshot before submitting.)



5. Challenges and How We Solved Them

- **No arrays or lists for items.** We stored the items in item1, item2, and item3 and used an if-else-if chain to fill the first empty slot; when all three are taken, addItem() prints a message and returns false.
- **The voucher with a no-argument receipt.** printReceipt() takes no parameters, yet Order #1002 needs a voucher. We added a private voucher field with a validated setVoucher() so the receipt can apply it while computeTotal(double) still guarantees the total never drops below 0.
- **Awarding points only once.** Printing a receipt twice would double the points, so Order keeps a flag and awards points only on the first print, based on the amount actually paid (205.00 gives 4 points).
- **Invalid data in constructors.** Calling the setters from the constructors means the same validation applies whether an object is built with a constructor or filled in later.
- **Matching the expected output.** We checked each figure by hand (for example 110 + 20 + 12 = 142 for the upsized Chicken Adobo) and adjusted formatting until the totals matched.